

## UNIT-1

### LEETCODE EXERCISE

#### 1. Big Countries

##### Problem Summary

You are given a table World with the following columns:

- name (country name)
- continent
- area (in square km)
- population
- GDB

A country is considered “big” if:

- Its area  $\geq 3,000,000$ , OR
- Its population  $\geq 25,000,000$

##### Task

Write an SQL query to return the name, population, and area of all big countries. **Solution**

 Code

MySQL   Auto

```
1
2 SELECT name, population, area
3 FROM World
4 WHERE area >= 3000000 OR population >= 25000000;
```

**Accepted** Runtime: 76 ms

 Case 1

Input

World =

| name        | continent | area    | population | gdp          |
|-------------|-----------|---------|------------|--------------|
| Afghanistan | Asia      | 652230  | 25500100   | 20343000000  |
| Albania     | Europe    | 28748   | 2831741    | 12960000000  |
| Algeria     | Africa    | 2381741 | 37100000   | 188681000000 |
| Andorra     | Europe    | 468     | 78115      | 3712000000   |
| Angola      | Africa    | 1246700 | 20609294   | 100990000000 |

Output

| name        | population | area    |
|-------------|------------|---------|
| Afghanistan | 25500100   | 652230  |
| Algeria     | 37100000   | 2381741 |

## Explanation

- The SELECT clause retrieves required columns.
- The WHERE clause filters records based on the given conditions.
- The OR operator ensures that even if one condition is satisfied, the country is included.

## 2 Find Customer Referee

### Problem Summary

You are given a table Customer with the following columns:

- id (customer ID)
- name (customer name)
- referee\_id (ID of the referring customer)

### Task

Find the names of customers who were NOT referred by the customer with id = 2.

- Note: Customers with NULL in referee\_id should also be included. **Solution**

### </> Code

MySQL Auto

```
1 SELECT name
2 FROM Customer
3 WHERE referee_id != 2 OR referee_id IS NULL;
```

Accepted Runtime: 81 ms

Case 1

Input

Customer =

| id | name | referee_id |
|----|------|------------|
| 1  | Will | null       |
| 2  | Jane | null       |
| 3  | Alex | 2          |
| 4  | Bill | null       |
| 5  | Zack | 1          |
| 6  | Mark | 2          |

Output

| name |
|------|
| Will |
| Jane |
| Bill |
| Zack |

### **Explanation**

- referee\_id = 2 → selects customers not referred by ID 2
- referee\_id IS NULL → includes customers with no referee
- OR ensures both conditions are considered

### **3. Not Boring Movies**

#### **Problem Summary**

You are given a table Cinema with the following columns:

- id (movie ID)
- movie (movie name)
- description (movie type)
- rating (movie rating)

#### **Task**

Find all movies that are:

- Not boring → description! = 'boring'
- Have odd-numbered IDs → id % 2 = 1

Return the result sorted by rating in descending order.

## Solution

 Code

MySQL   Auto

```
1 SELECT *
2 FROM Cinema
3 WHERE description != 'boring'
4 AND id % 2 = 1
5 ORDER BY rating DESC;
```

Accepted Runtime: 86 ms

 Case 1

Input

cinema =

| id | movie      | description | rating |
|----|------------|-------------|--------|
| 1  | War        | great 3D    | 8.9    |
| 2  | Science    | fiction     | 8.5    |
| 3  | irish      | boring      | 6.2    |
| 4  | Ice song   | Fantasy     | 8.6    |
| 5  | House card | Interesting | 9.1    |

Output

| id | movie      | description | rating |
|----|------------|-------------|--------|
| 5  | House card | Interesting | 9.1    |
| 1  | War        | great 3D    | 8.9    |

## Explanation

- `description != 'boring'` → filters out boring movies
- `id % 2 = 1` → selects only odd-numbered IDs
- `ORDER BY rating DESC` → sorts result from highest to lowest rating

## 4. Invalid Tweets Problem Summary

You are given a table Tweets with the following column:

- `tweet_id` (ID of the tweet)
- `content` (text of the tweet)

## Task

Find the IDs of tweets that are invalid, where a tweet is considered invalid if:

- The length of the content is greater than 15 characters **Solution**

</> Code

MySQL   Auto

```
1 SELECT tweet_id
2 FROM Tweets
3 WHERE LENGTH(content) > 15;
```

Accepted Runtime: 63 ms

 Case 1

Input

Tweets =

| tweet_id | content                           |
|----------|-----------------------------------|
| 1        | Let us Code                       |
| 2        | More than fifteen chars are here! |

Output

| tweet_id |
|----------|
| 2        |

## Explanation

- LENGTH (content) calculates the number of characters in the tweet
- 15 filters tweets that exceed the allowed length
- Only tweet\_id is returned as required

## 5. Recyclable and Low-Fat Products Problem Summary

You are given a table Products with the following columns:

- product\_id
- low\_fats (values: 'Y' or 'N')
- recyclable (values: 'Y' or 'N')

## Task

Find the product IDs of products that are:

- Low fat (low\_fats = 'Y')

- Recyclable (recyclable = 'Y')

## Solution

 Code

MySQL   Auto

```
1 SELECT product_id
2 FROM Products
3 WHERE low_fats = 'Y'
4 AND recyclable = 'Y';
```

Accepted Runtime: 101 ms

 Case 1

Input

Products =

| product_id | low_fats | recyclable |
|------------|----------|------------|
| 0          | Y        | N          |
| 1          | Y        | Y          |
| 2          | N        | Y          |
| 3          | Y        | Y          |
| 4          | N        | N          |

Output

| product_id |
|------------|
| 1          |
| 3          |

## Explanation

- low\_fats = 'Y' → selects only low-fat products
- recyclable = 'Y' → selects recyclable products
- AND ensures both conditions are satisfied

## 6.Article Views I

### Problem Summary

You are given a table Views with the following columns:

- article\_id
- author\_id
- viewer\_id
- view\_id

Task Find all authors who viewed at least one of their own articles.

- Return the result as a list of unique author IDs
- Sort the result in ascending order

## Solution

 Code

MySQL   Auto

```
1 SELECT DISTINCT author_id AS id
2 FROM Views
3 Where author_id = viewer_id
4 ORDER BY id ASC;
```

Accepted Runtime: 81 ms

 Case 1

Input

Views =

| article_id | author_id | viewer_id | view_date  |
|------------|-----------|-----------|------------|
| 1          | 3         | 5         | 2019-08-01 |
| 1          | 3         | 6         | 2019-08-02 |
| 2          | 7         | 7         | 2019-08-01 |
| 2          | 7         | 6         | 2019-08-02 |
| 4          | 7         | 1         | 2019-07-22 |
| 3          | 4         | 4         | 2019-07-21 |

Output

| id |
|----|
| 4  |
| 7  |

## Explanation

- `author_id = viewer_id` → identifies cases where authors viewed their own articles
- `DISTINCT` → removes duplicate author IDs
- `ORDER BY id` → sorts the result in ascending order